

INTERNSHIP PROGRESS DASHBOARD

Technical & Testing Document

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Computer Science & Engineering

Submitted by

[Utkarsh Dwivedi]

AKS UNIVERSITY

Semester: [VIII]

Under the Guidance of

[Mentor / Guide Name]

[MP ONLINE]

[College / University Name]

AKS UNIVERSITY

Department of Computer Science & Engineering

[satna, mp]

Academic Year: 2025 – 2026

1. Project Overview

The Internship Progress Dashboard is a full-stack web application that enables mentors to manage, assign, and track tasks for their assigned interns (students). Students can submit weekly reports, update task statuses, and receive feedback — all in one place.

Component	Technology	Details
Backend Framework	FastAPI (Python)	Async REST API with auto docs at /api/docs
Database	MongoDB + Beanie ODM	Document-based async ORM
Authentication	JWT (Bearer Token)	HS256 signing, role embedded in token
Frontend	React + Vite	Served separately on port 5173
Containerization	Docker	Dockerfile present for backend
Notifications	In-app (MongoDB)	Real-time read/unread tracking

1.1 User Roles

The system has exactly two roles. Admin and Mentor are the same single role — there is no separate admin. Every mentor acts as admin within their student scope.

Role	Description	Key Permissions
mentor	Instructor / Admin (same entity)	Create tasks, assign students, view analytics, read all reports of own students
student	Intern	Update own task status, submit weekly reports, view own analytics, receive notifications

2. System Architecture

2.1 Backend Structure

Path	Responsibility
app/main.py	FastAPI app init, CORS middleware, router registration
app/config.py	Environment variables (MONGO_URL, SECRET_KEY, etc.)
app/database.py	Beanie + Motor async MongoDB connection & init
app/models/	Beanie Document models: User, Task, Report, Notification
app/routes/auth.py	Register, Login, /me endpoints

Path	Responsibility
app/routes/tasks.py	CRUD for tasks, role-based access
app/routes/reports.py	CRUD for weekly reports + comment system
app/routes/analytics.py	Dashboard analytics (different per role)
app/routes/users.py	Student listing, assign/unassign toggle, profile update
app/routes/notifications.py	Fetch, mark-read, mark-all-read
app/utils/auth.py	JWT create/verify, password hash/verify, get_current_user

2.2 Data Models

User

Field	Type	Notes
name	str	Required
email	EmailStr	Unique — validated at registration
password	str	Hashed (bcrypt)
role	Enum: student mentor	Default: student
mentor_id	Optional[str]	Student only — assigned mentor's ObjectId
skills	List[str]	Empty by default
avatar, bio	Optional[str]	Profile fields

Task

Field	Type	Notes
title	str	Required
status	Enum: todo in_progress completed	Default: todo
priority	Enum: low medium high	Default: medium
assigned_to	str	Student ObjectId
created_by	str	Mentor ObjectId
due_date	Optional[datetime]	ISO 8601 string in API
tags	List[str]	Searchable tags

Report

Field	Type	Notes
title	str	Required
content	str	Markdown text
student_id	str	Author ObjectId

Field	Type	Notes
week_number	int	Internship week number
comments	List[Comment]	Embedded — not a separate collection

3. API Endpoint Reference

3.1 Authentication [/api/auth]

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Create new user account
POST	/api/auth/login	Public	Login and receive JWT token
GET	/api/auth/me	Any Auth	Get current user profile

3.2 Tasks [/api/tasks]

Method	Endpoint	Access	Description
GET	/api/tasks/	Any Auth	List tasks (mentor: created by me; student: assigned to me)
POST	/api/tasks/	Mentor	Create new task, auto-notify assigned student
PUT	/api/tasks/{id}	Both	Mentor: full edit; Student: status update only
DELETE	/api/tasks/{id}	Mentor	Delete own task

3.3 Reports [/api/reports]

Method	Endpoint	Access	Description
GET	/api/reports/	Any Auth	Mentor: reports from own students; Student: own reports
GET	/api/reports/{id}	Any Auth	Single report (student restricted to own)
POST	/api/reports/	Student	Submit weekly report, notify mentor
PUT	/api/reports/{id}	Student	Edit own report
DELETE	/api/reports/{id}	Student	Delete own report
POST	/api/reports/{id}/comments	Any Auth	Add comment, notify student if mentor
DELETE	/api/reports/{id}/comments/{cid}	Any Auth	Delete own comment only

3.4 Analytics [/api/analytics]

Method	Endpoint	Access	Description
GET	/api/analytics/	Any Auth	Role-specific dashboard data (different schema per role)

Mentor response includes: total_students, task_status breakdown, completion_rate, weekly_activity (7 weeks), student_progress per student.

Student response includes: total_tasks, total_reports, task_status, completion_rate, weekly_activity, priority_breakdown, feedback_received (comment count).

3.5 Users [/api/users]

Method	Endpoint	Access	Description
GET	/api/users/students	Mentor	List own assigned students
GET	/api/users/all-students	Mentor	List all students (for assign/unassign UI)
PUT	/api/users/assign-toggle/{id}	Mentor	Toggle assign/unassign student to self
PUT	/api/users/profile	Any Auth	Update name, bio, skills, avatar

3.6 Notifications [/api/notifications]

Method	Endpoint	Access	Description
GET	/api/notifications/	Any Auth	Last 30 notifications for current user
GET	/api/notifications/unread-count	Any Auth	Returns { count: N }
PUT	/api/notifications/{id}/read	Any Auth	Mark single notification as read
PUT	/api/notifications/read-all	Any Auth	Mark all user notifications as read

4. Test Cases

These test cases cover the full system. Test IDs follow the format: MODULE-TC-NNN.

4.1 Authentication Tests

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
AUTH-TC-001	Register Student	POST /register with valid name, email, password, role=student	201 / JWT token returned, user in DB	
AUTH-TC-002	Register Mentor	POST /register with valid data, role=mentor	201 / JWT token returned	

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
AUTH-TC-003	Duplicate Email	POST /register with already-used email	400 / 'Email already registered'	
AUTH-TC-004	Login Valid	POST /login with correct credentials	200 / token + user object	
AUTH-TC-005	Login Wrong Password	POST /login with wrong password	401 / 'Invalid email or password'	
AUTH-TC-006	Login Unknown Email	POST /login with unregistered email	401 / Unauthorized	
AUTH-TC-007	Get Me — Valid Token	GET /me with Bearer token	200 / current user profile	
AUTH-TC-008	Get Me — No Token	GET /me without Authorization header	401 / Unauthorized	
AUTH-TC-009	Get Me — Expired Token	GET /me with expired JWT	401 / Unauthorized	

4.2 Task Tests

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
TASK-TC-001	Mentor Creates Task	POST /tasks/ as mentor with valid payload	201 / task object, notification created for student	
TASK-TC-002	Student Cannot Create Task	POST /tasks/ as student	403 / 'Only mentors can create tasks'	
TASK-TC-003	Mentor Gets Own Tasks	GET /tasks/ as mentor	200 / only tasks created_by mentor	
TASK-TC-004	Student Gets Own Tasks	GET /tasks/ as student	200 / only tasks assigned_to student	
TASK-TC-005	Filter by Status	GET /tasks/?status=completed	200 / only completed tasks returned	
TASK-TC-006	Search Tasks	GET /tasks/?search=python	200 / tasks matching title/desc/tags	
TASK-TC-007	Mentor Updates Task	PUT /tasks/{id} as task creator (mentor)	200 / all fields updated	
TASK-TC-008	Student Updates Status	PUT /tasks/{id} with {status:completed} as assigned student	200 / status changed, other fields unchanged	
TASK-TC-009	Student Cannot Edit Title	PUT /tasks/{id} with {title:'New'} as student	200 / title NOT changed (only status respected)	
TASK-TC-010	Mentor Deletes Task	DELETE /tasks/{id} as creator mentor	200 / 'Task deleted successfully'	
TASK-TC-011	Student Cannot Delete Task	DELETE /tasks/{id} as student	403 / 'Only mentors can delete tasks'	

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
TASK-TC-012	Task Not Found	GET/PUT/DELETE with invalid Objectid	404 / 'Task not found'	
TASK-TC-013	Cross-Mentor Access Denied	Mentor tries to edit another mentor's task	403 / 'Access denied'	

4.3 Report Tests

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
RPT-TC-001	Student Submits Report	POST /reports/ as student with title, content, week_number	201 / report created, mentor notified	
RPT-TC-002	Mentor Cannot Submit Report	POST /reports/ as mentor	403 / 'Only students can submit reports'	
RPT-TC-003	Student Views Own Reports	GET /reports/ as student	200 / own reports only	
RPT-TC-004	Mentor Views Assigned Reports	GET /reports/ as mentor	200 / reports from own students only	
RPT-TC-005	Report Not Found	GET /reports/{invalid-id}	404 / 'Report not found'	
RPT-TC-006	Student Updates Report	PUT /reports/{id} as report owner	200 / updated report	
RPT-TC-007	Student Deletes Report	DELETE /reports/{id} as report owner	200 / 'Report deleted'	
RPT-TC-008	Mentor Adds Comment	POST /reports/{id}/comments as mentor	200 / comment added, student notified	
RPT-TC-009	Student Adds Comment	POST /reports/{id}/comments as student	200 / comment added (no notification)	
RPT-TC-010	Delete Own Comment	DELETE /reports/{id}/comments/{cid} as comment author	200 / comment removed	
RPT-TC-011	Cannot Delete Others Comment	DELETE /reports/{id}/comments/{cid} as different user	404 / 'Comment not found or not authorized'	
RPT-TC-012	No Notification if No Mentor	Student submits report with mentor_id=None	201 / created, no notification insert error	

4.4 Analytics Tests

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
ANA-TC-001	Mentor Analytics — Basic	GET /analytics/ as mentor with assigned students + tasks	role=mentor, total_students, task_status, completion_rate present	
ANA-TC-002	Mentor Analytics — No Students	GET /analytics/ as mentor with no assigned students	student_progress=[], total_students=0	
ANA-TC-003	Student Analytics — Basic	GET /analytics/ as student with tasks and reports	role=student, weekly_activity, priority_breakdown, feedback_received present	
ANA-TC-004	Student Analytics — Empty	GET /analytics/ as student with zero tasks/reports	completion_rate=0.0, totals all 0	
ANA-TC-005	Weekly Activity — 7 Weeks	Any role after creating tasks across multiple weeks	weekly_activity array has exactly 7 items	
ANA-TC-006	Per-Student Progress Rate	Mentor has 1 student with 4 tasks, 2 completed	student_progress[0].rate = 50.0	

4.5 User Management Tests

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
USR-TC-001	Mentor Gets Own Students	GET /users/students as mentor	200 / only students with mentor_id=me	
USR-TC-002	Get All Students	GET /users/all-students as mentor	200 / all students regardless of mentor_id	
USR-TC-003	Student Cannot List Students	GET /users/students as student	403 / 'Access denied'	
USR-TC-004	Assign Student (Toggle ON)	PUT /assign-toggle/{id} — student has no mentor_id	200 / 'Student Assigned', mentor_id set	
USR-TC-005	Unassign Student (Toggle OFF)	PUT /assign-toggle/{id} — student already has mentor_id	200 / 'Student Unassigned', mentor_id=None	
USR-TC-006	Assign Non-existent Student	PUT /assign-toggle/{invalid-id}	404 / 'Student not found'	
USR-TC-007	Update Own Profile	PUT /users/profile with name, bio, skills	200 / updated profile object	
USR-TC-008	Profile — Partial Update	PUT /users/profile with only skills field	200 / only skills updated, rest unchanged	

4.6 Notification Tests

Test ID	Test Name	Input / Action	Expected Result	Pass/Fail
NOTIF-TC-001	Task Creation Notifies Student	Mentor creates task assigned to student	Notification in DB with type=task_assigned for student	
NOTIF-TC-002	Report Submission Notifies Mentor	Student submits report (has mentor_id)	Notification for mentor with type=report_submitted	
NOTIF-TC-003	Mentor Comment Notifies Student	Mentor comments on student's report	Notification for student with type=feedback_received	
NOTIF-TC-004	Get Notifications	GET /notifications/ as any user	200 / max 30, sorted newest first	
NOTIF-TC-005	Unread Count	GET /notifications/unread-count before reading any	{ count: N } where N = unread count	
NOTIF-TC-006	Mark One Read	PUT /notifications/{id}/read	200 / notification.read=true in DB	
NOTIF-TC-007	Mark All Read	PUT /notifications/read-all	200 / all user's notifications read=true	
NOTIF-TC-008	Cannot Read Others Notifications	PUT /notifications/{other-user-notif-id}/read	200 / silently ignored (no change)	

5. Known Issues & Gaps

#	Issue	Severity	Recommendation
1	CORS allows all origins (*) — development only	Medium	Restrict to known frontend URL in production
2	No pagination on /tasks/, /reports/, or analytics	Medium	Add limit/offset or cursor-based pagination
3	No refresh token — JWT expires without seamless renewal	Medium	Implement refresh token endpoint
4	assign-toggle has no multi-mentor conflict check — student can be re-assigned away from another mentor	High	Check if student.mentor_id belongs to a different mentor before overwriting
5	Report access check for mentor is missing in GET /{report_id} — any auth can access	High	Add mentor ownership check: student must be in mentor's assigned list
6	datetime.utcnow() is deprecated in Python 3.12+	Low	Use datetime.now(timezone.utc) instead

#	Issue	Severity	Recommendation
7	No email validation or password strength enforcement beyond Pydantic EmailStr	Low	Add min-length and complexity rules for password
8	mark_all_read does individual saves in a loop — N+1 DB calls	Low	Use bulk update: Notification.find(...).update({'\$set': {'read': True}})
9	No file upload support (avatar field is string URL only)	Low	Add S3/local file upload if real avatars needed
10	No rate limiting on /register and /login	Medium	Add slowapi or similar middleware

6. Setup & Local Run

6.1 Environment Variables (.env)

Variable	Example Value	Description
MONGO_URL	mongodb://localhost:27017	MongoDB connection string
DATABASE_NAME	internship_db	Database name
SECRET_KEY	your-secret-key-here	JWT signing key (keep secret!)
ACCESS_TOKEN_EXPIRE_MINUTES	1440	Token lifetime in minutes (1 day)

6.2 Run Without Docker

- `cd backend && python -m venv venv`
- `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Linux/Mac)
- `pip install -r requirements.txt`
- Copy `.env.example` to `.env` and fill values
- `uvicorn app.main:app --reload --port 8000`
- API Docs available at: <http://localhost:8000/api/docs>

6.3 Run With Docker

- `docker build -t internship-backend ./backend`
- `docker run -p 8000:8000 --env-file backend/.env internship-backend`

6.4 Seed Database

- `python backend/seed.py` — creates sample mentor, students, tasks, reports

7. Manual Testing Checklist

Use Postman, Insomnia, or the built-in FastAPI Swagger UI at `/api/docs`.

Step	Action	Done?
1	Register a mentor account	
2	Register 2 student accounts	
3	Login as mentor — save token	
4	Assign both students via <code>/api/users/assign-toggle/{id}</code>	
5	Create 3 tasks (different priorities) assigned to student 1	
6	Verify student 1 gets 3 notifications	
7	Login as student 1 — update one task to <code>in_progress</code>	
8	Student 1 submits a weekly report (week 1)	
9	Mentor logs in — verify report notification exists	
10	Mentor adds a comment to student 1's report	
11	Student 1 checks notifications — should see <code>feedback_received</code>	
12	Check <code>/api/analytics/</code> as mentor — verify <code>student_progress</code> data	
13	Check <code>/api/analytics/</code> as student — verify <code>completion_rate</code>	
14	Mentor marks all notifications as read — verify <code>unread-count=0</code>	
15	Try DELETE on another user's task — expect 403	

8. Document Sign-Off

Prepared By	Reviewed By	Version	Date
Development Team	Mentor / Admin	1.0	April 2026

This document should be updated whenever a new API endpoint is added, a model changes, or a critical bug is discovered.